Introduction to Algorithms — A Short Course

Sample book for the AIB Graph demo

Foundations of Algorithms

An algorithm is a finite, well-defined procedure that transforms an input into an output. This chapter introduces the vocabulary used in the rest of the course: problems and instances, pseudocode, correctness and the loop invariant technique used to prove that iterative algorithms do what they claim.

What Is an Algorithm

A computational problem specifies a relationship between inputs and outputs, for example: given a sequence of numbers, return the same numbers in non-decreasing order. Each concrete input, such as (31, 41, 59, 26), is an instance of the problem. An algorithm is a procedure that solves every instance: it is made of a finite number of unambiguous steps, it always halts, and it produces the correct output. Algorithms are independent of programming languages; the same algorithm can be written in Python, C or plain English. What matters is that each step is precise and can be executed mechanically. Efficiency is the second concern: two correct algorithms for the same problem can differ enormously in the time and memory they need, and that difference often matters more than faster hardware.

- A problem relates inputs to outputs; an instance is one concrete input
- An algorithm is finite, unambiguous and always halts
- Correct algorithms can still differ greatly in efficiency

Pseudocode and Correctness

Pseudocode describes an algorithm with the structure of a programming language (assignments, conditionals, loops and procedure calls) while hiding details such as memory management or error handling. It lets us reason about the algorithm itself. An algorithm is correct if, for every input instance, it halts with the correct output. An incorrect algorithm may fail to halt on some inputs or may halt with a wrong answer. Testing on examples can reveal bugs, but it cannot prove correctness, because the number of instances is usually infinite. Proofs of correctness therefore reason about all inputs at once, usually through invariants for loops and induction for recursion. A correct but extremely slow algorithm can still be useless in practice, which is why correctness and efficiency are always studied together.

- Pseudocode keeps the structure and hides implementation details
- Correct means: halts with the right output on every instance
- Tests find bugs; proofs establish correctness

Loop Invariants

A loop invariant is a property that holds before every iteration of a loop. To use it as a proof of correctness we show three things. Initialization: the invariant is true before the first iteration. Maintenance: if it is true before an iteration, it remains true before the next one. Termination: when the loop ends, the invariant, together with the reason the loop stopped, gives a useful property that shows the algorithm is correct. The technique resembles mathematical induction, except that the induction stops when the loop terminates. For insertion sort, the invariant is: at the start of each iteration, the subarray to the left of the current element consists of the original elements of that part, but in sorted order. When the loop ends, that subarray is the whole array, so the array is sorted.

- Three steps: initialization, maintenance, termination
- Loop invariants are induction that stops when the loop ends
- Insertion sort's invariant: the left part is always sorted

Asymptotic Analysis

Exact running times depend on hardware and implementation details. Asymptotic analysis ignores those constant factors and asks how the running time grows as the input size n becomes large. This chapter introduces the notations used to express those growth rates and the techniques to compute them for loops and recursive algorithms.

Big-O Notation

We write f(n) = O(g(n)) when there are positive constants c and n0 such that f(n) <= c * g(n) for every n >= n0. In words, beyond some input size, f grows no faster than a constant multiple of g. Big-O is an upper bound: it promises that the running time will not be worse than the bound for large inputs. Constant factors and lower-order terms disappear, so 3n^2 + 10n + 7 is O(n^2). Common growth rates, from slowest to fastest, are O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n). When we say an algorithm runs in O(n^2) time we usually mean its worst-case running time is bounded by a quadratic function of the input size.

- f(n) = O(g(n)) means f grows no faster than c * g(n) for large n
- Constants and lower-order terms are dropped
- Common classes: 1, log n, n, n log n, n^2, 2^n

Big-Omega and Big-Theta

Big-Omega is the mirror image of Big-O: f(n) = Omega(g(n)) when f(n) >= c * g(n) for some positive constant c and all large enough n, so g is an asymptotic lower bound. Big-Theta combines both: f(n) = Theta(g(n)) when f is both O(g(n)) and Omega(g(n)), so g describes the growth of f exactly, up to constant factors. Merge sort runs in Theta(n log n) time on every input. Insertion sort runs in O(n^2) time, but it is not Theta(n^2) on every input: on an already sorted array it takes only Theta(n). Lower bounds can also describe problems rather than algorithms: any comparison-based sorting algorithm needs Omega(n log n) comparisons in the worst case.

- Omega is a lower bound, Theta is a tight bound
- Theta means both O and Omega of the same function
- Comparison sorting needs Omega(n log n) comparisons in the worst case

Analyzing Loops and Recursion

For iterative code we count how many times the innermost operation runs. A single loop over n elements does Theta(n) work; two nested loops over n elements do Theta(n^2). For recursive algorithms we write a recurrence that expresses the running time on input size n in terms of smaller inputs. Merge sort satisfies T(n) = 2T(n/2) + Theta(n): two half-size subproblems plus linear work to merge. A recursion tree makes the sum visible: each level of the tree does Theta(n) work in total and there are about log2 n levels, so T(n) = Theta(n log n). Binary search satisfies T(n) = T(n/2) + Theta(1), which gives Theta(log n). The master theorem packages these cases into a general rule for recurrences of the form T(n) = aT(n/b) + f(n).

- Nested loops multiply their iteration counts
- Recursive running times are written as recurrences
- T(n) = 2T(n/2) + n solves to Theta(n log n)

Sorting

Sorting is the most studied problem in algorithms: it is useful on its own and it is a building block for many other algorithms. This chapter compares an incremental algorithm (insertion sort) with two divide-and-conquer algorithms (merge sort and quicksort) and analyzes their running times.

Insertion Sort

Insertion sort works the way many people sort a hand of playing cards. It keeps a sorted prefix of the array and, at each step, takes the next element (the key) and shifts larger elements of the prefix one position to the right until the key's position is found. It sorts in place, using only a constant amount of extra memory, and it is stable: equal elements keep their relative order. In the worst case, an array sorted in reverse, each key is compared with every element of the prefix, giving Theta(n^2) time. In the best case, an already sorted array, it takes Theta(n). Because of its low overhead, insertion sort is often used for small arrays inside faster algorithms.

- Grows a sorted prefix by inserting one key at a time
- In place and stable
- Theta(n^2) worst case, Theta(n) on sorted input

Merge Sort

Merge sort follows the divide-and-conquer paradigm. Divide: split the array into two halves. Conquer: sort each half recursively. Combine: merge the two sorted halves into one sorted array. Merging walks through both halves with two pointers, always copying the smaller front element, so it takes Theta(n) time. The recurrence T(n) = 2T(n/2) + Theta(n) gives Theta(n log n) in every case: best, average and worst. Merge sort is stable, but the standard array version needs Theta(n) extra memory for the merge step. It works well on linked lists and on data too large to fit in memory, where its sequential access pattern is an advantage.

- Divide, conquer, combine
- Merging two sorted halves takes linear time
- Always Theta(n log n); needs Theta(n) extra memory

Quicksort

Quicksort also uses divide and conquer, but it does the hard work before recursing. It picks a pivot element and partitions the array so that elements smaller than the pivot come before it and larger elements come after it. The pivot is then in its final position, and quicksort recursively sorts the two sides. Partitioning takes Theta(n) time and happens in place. If the pivot always splits the array evenly, the running time is Theta(n log n). If the pivot is always the smallest or largest element, for example when the first element is chosen on an already sorted array, the split is maximally unbalanced and the running time degrades to Theta(n^2). Choosing the pivot at random makes the expected running time Theta(n log n) on every input. In practice quicksort is often the fastest comparison sort because of its small constant factors and good cache behaviour.

- Partition around a pivot, then recurse on both sides
- In place; Theta(n log n) expected with a random pivot
- Bad pivots give Theta(n^2)

Data Structures

An algorithm is only as fast as the operations it performs on its data. A data structure organizes data to support a set of operations efficiently, such as insert, delete and search. This chapter presents three families: linear structures with restricted access (stacks and queues), hash tables for fast lookup by key, and binary search trees for ordered data.

Stacks and Queues

A stack is a last-in, first-out (LIFO) collection: push adds an element on top and pop removes the most recently added one. A queue is first-in, first-out (FIFO): enqueue adds at the tail and dequeue removes from the head. Both can be implemented with an array or a linked list so that every operation takes O(1) time. Stacks appear in recursion (the call stack), expression evaluation and undo features. Queues model waiting lines, buffers between producers and consumers, and they are the core of breadth-first search. A circular array implementation of a queue reuses the free slots at the start of the array, wrapping the head and tail indices around.

- Stack: LIFO, push and pop
- Queue: FIFO, enqueue and dequeue
- All operations take O(1) time

Hash Tables

A hash table stores key-value pairs in an array of buckets. A hash function maps each key to a bucket index. When two keys hash to the same bucket we have a collision. With chaining, each bucket holds a linked list of the pairs that hash there. With open addressing, a colliding key probes other slots of the array until it finds a free one. If the hash function spreads keys uniformly and the load factor (number of keys divided by number of buckets) stays bounded, insert, delete and search take O(1) expected time. When the load factor grows too large the table is resized, typically doubling, and every key is rehashed. Hash tables do not keep keys in order, so operations such as finding the minimum key take O(n) time.

- A hash function maps keys to buckets
- Collisions are handled by chaining or open addressing
- O(1) expected time with a bounded load factor; no ordering

Binary Search Trees

A binary search tree stores keys in nodes so that, for every node, keys in its left subtree are smaller and keys in its right subtree are larger. Searching starts at the root and goes left or right at each node, so it takes O(h) time, where h is the height of the tree. Insertion follows the same path and adds the new key as a leaf. An in-order traversal visits the keys in sorted order in Theta(n) time, and operations such as minimum, maximum, successor and range queries are natural. The weakness is the height: inserting keys in sorted order produces a tree that is just a long path, with h = n. Balanced trees such as red-black trees and AVL trees restructure themselves to keep h = O(log n).

- Left subtree smaller, right subtree larger
- Search and insert take O(h) time
- Balanced trees keep h = O(log n)

Graph Algorithms

Graphs model relationships: road networks, social networks, dependencies between tasks, links between web pages. A graph G = (V, E) has a set of vertices V and a set of edges E. This chapter shows how to store graphs, how to explore them systematically with breadth-first search and how to find shortest paths when edges have weights.

Graph Representations

There are two standard ways to store a graph G = (V, E). An adjacency list keeps, for each vertex, a list of its neighbours. It uses Theta(V + E) space and lets us iterate over the neighbours of a vertex in time proportional to its degree, which is ideal for sparse graphs, where E is much smaller than V^2. An adjacency matrix is a V x V table whose entry (u, v) is 1 when there is an edge from u to v. It uses Theta(V^2) space but answers the question 'is there an edge between u and v?' in O(1) time, which suits dense graphs. Weighted graphs store the weight in the list entry or the matrix cell. Directed graphs store each edge once; undirected graphs store it in both directions.

- Adjacency list: Theta(V + E) space, good for sparse graphs
- Adjacency matrix: Theta(V^2) space, O(1) edge test
- Weights are stored alongside each edge

Breadth-First Search

Breadth-first search (BFS) explores a graph from a source vertex s in waves. It first visits all neighbours of s, then all vertices two edges away, and so on. A queue holds the discovered vertices that still need to be explored, which guarantees that vertices are processed in order of their distance from s. Each vertex is marked when discovered so it is enqueued only once. With an adjacency list, BFS runs in O(V + E) time. As a by-product it computes, for every reachable vertex, the minimum number of edges from s and a predecessor pointer; following predecessors back to s gives a shortest path. BFS therefore solves the shortest path problem in unweighted graphs, but not when edges have different weights.

- Explores vertices in order of distance from the source
- Uses a queue; O(V + E) time
- Gives shortest paths in unweighted graphs

Shortest Paths with Dijkstra

Dijkstra's algorithm finds shortest paths from a source s in a graph whose edge weights are non-negative. It keeps a tentative distance d[v] for every vertex, starting with d[s] = 0 and infinity elsewhere. At each step it removes from a priority queue the unsettled vertex u with the smallest d[u]; that distance is now final. It then relaxes every edge (u, v): if d[u] + w(u, v) < d[v], it lowers d[v] and records u as v's predecessor. With a binary heap the algorithm runs in O((V + E) log V) time. The greedy choice is safe only because weights are non-negative: a later path cannot become shorter by adding more edges. With negative weights, the Bellman-Ford algorithm must be used instead.

- Settles the closest unsettled vertex at every step
- Relaxation: d[v] = min(d[v], d[u] + w(u, v))
- Needs non-negative weights; O((V + E) log V) with a heap